

```

import torch
import torch.nn as nn
import torchvision.transforms as transforms
from PIL import Image
import matplotlib.pyplot as plt
import os

#image
image_path = "046.jpg"
if not os.path.exists(image_path):
    print("ERROR: Image not found. Available files:")
    print(os.listdir())
    raise SystemExit

image = Image.open(image_path).convert("L")

# Converts image to a PyTorch tensor and Normalizes pixel values to [0, 1]
transform = transforms.ToTensor()
img_tensor = transform(image).unsqueeze(0)

# Defines a CNN class
# Input channels = 1 , Ouput channels = 2 , Kernel size = 3x3
class EdgeCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 2, 3, bias=False)

        Kh = torch.tensor([
            [-1,-1,-1],
            [ 0, 0, 0],
            [ 1, 1, 1]
        ], dtype=torch.float32)

        Kv = torch.tensor([
            [-1, 0, 1],
            [-1, 0, 1],
            [-1, 0, 1]
        ], dtype=torch.float32)

        self.conv.weight.data = torch.stack([Kh, Kv]).unsqueeze(1)

        # This false ensures that Kernels are fixed
        self.conv.weight.requires_grad = False

        # This function applies convolution to the input image.
        def forward(self, x):
            return self.conv(x)

model = EdgeCNN()
output = model(img_tensor)

plt.figure(figsize=(12,4))

# Original Image
plt.subplot(1,3,1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Horizontal Edges
plt.subplot(1,3,2)
plt.imshow(output[0,0].detach(), cmap="gray")
plt.title("Horizontal Edges")
plt.axis("off")

# Vertical Edges
plt.subplot(1,3,3)
plt.imshow(output[0,1].detach(), cmap="gray")
plt.title("Vertical Edges")
plt.axis("off")

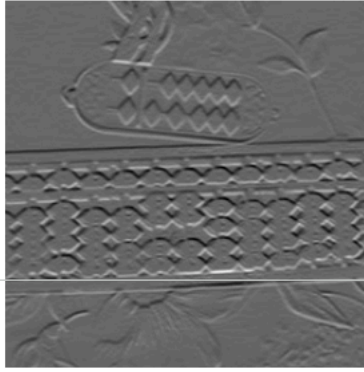
plt.show()

```

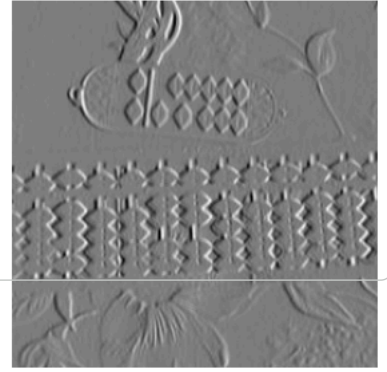
Original Image



Horizontal Edges



Vertical Edges



```

# libraries
import torch
import torch.nn as nn
import torchvision.transforms as transforms
from PIL import Image
import matplotlib.pyplot as plt
import os

# image
image_path = "046.jpg"
if not os.path.exists(image_path):
    print("ERROR: Image not found")
    print("Available files:", os.listdir())
    raise SystemExit

image = Image.open(image_path).convert("RGB")

# Resizes image
transform = transforms.Compose([
    transforms.Resize((64,64)),
    transforms.ToTensor()
])

images = transform(image).unsqueeze(0)

# Definition of CNN
class DeepCNN(nn.Module):
    def __init__(self, activation):
        super().__init__()

        #convolution blocks
        self.blocks = nn.ModuleList([
            nn.Sequential(nn.Conv2d(3,32,3,padding=1), nn.ReLU()),
            nn.Sequential(nn.Conv2d(32,32,3,padding=1), nn.ReLU()),
            nn.Sequential(nn.Conv2d(32,32,3,padding=1), nn.ReLU()),
            nn.Sequential(nn.Conv2d(32,32,3,padding=1), nn.ReLU()),
            nn.Sequential(nn.Conv2d(32,32,3,padding=1), nn.ReLU())
        ])

        # Converts 3D feature - 1D vector
        self.flatten = nn.Flatten()
        self.fc = nn.Linear(32*64*64, 10)
        self.activation = activation

    def forward(self, x):
        feature_maps = []
        for block in self.blocks:
            x = block(x)
            feature_maps.append(x)
        x = self.flatten(x)
        x = self.fc(x)
        return self.activation(x), feature_maps

# Activation Comparison of ReLU, LeakyReLU,Tanh
activations = {
    "ReLU": nn.ReLU(),
    "LeakyReLU": nn.LeakyReLU(0.01),
    "Tanh": nn.Tanh()
}

for name, act in activations.items():
    print("\nActivation:", name)
    model = DeepCNN(act)

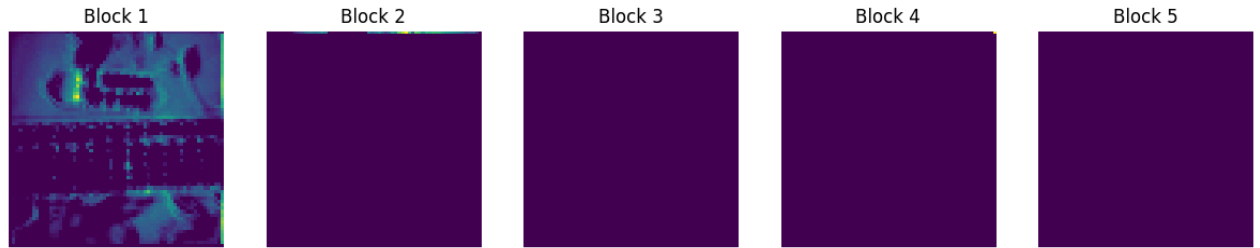
    output, features = model(images)
    print("Output:", output.detach().numpy())

    plt.figure(figsize=(15,3))
    for i in range(5):
        plt.subplot(1,5,i+1)
        plt.imshow(features[i][0,0].detach())
        plt.title(f"Block {i+1}")
        plt.axis("off")
    plt.show()

```

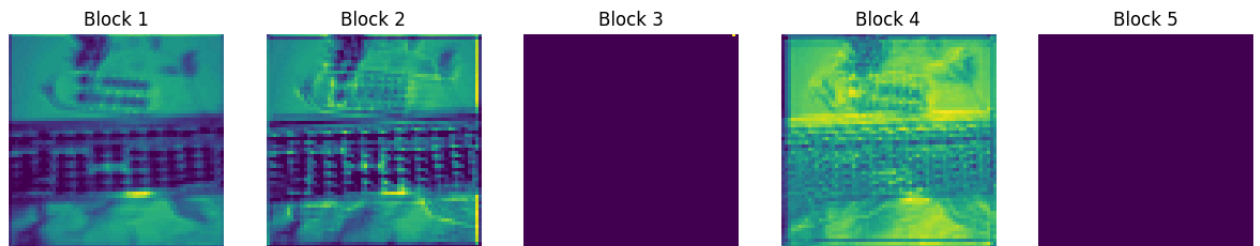
Activation: ReLU

Output: $\begin{bmatrix} 0.00115349 & 0. & 0. & 0.00426123 & 0. & 0.00084029 \\ 0. & 0.00333077 & 0. & 0. & 0. & 0. \end{bmatrix}$



Activation: LeakyReLU

Output: $\begin{bmatrix} 1.3023458e-02 & 6.2502110e-03 & -1.2115156e-04 & -2.6069695e-04 \\ -1.9809938e-04 & 1.3003351e-02 & -3.4071447e-05 & -1.5939413e-05 \\ 2.5444835e-02 & -3.8805720e-05 \end{bmatrix}$



Activation: Tanh

Output: $\begin{bmatrix} 0.009484 & -0.00199104 & 0.0025329 & -0.0143585 & -0.00724878 & 0.01207817 \\ -0.02916797 & -0.01606224 & -0.00609998 & -0.00540034 \end{bmatrix}$

